

OPTIMALITY OF TD(0).

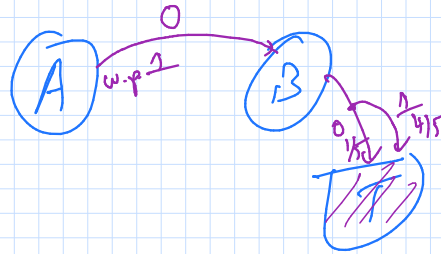
→ TD(0) results in "certainly-equivalent estimates" of the value function, given the data.

↳ If model captured by the data is accurate, TD(0) estimates converge to the true value function.

→ MC based estimate minimizes the squared error given the data.

1st: A, 0, B, 0
2nd: B, 1
3rd: B, 1
4th: B, 1
5th: B, 1

} 5 episodes of data.



MC updates? $V_\pi(B) = \frac{4}{5}$, $V_\pi(A) = 0$

TD updates? → First Ep: $V_\pi(A) = 0$, $V_\pi(B) = 0$

2nd: $V_\pi(A) = 0$, $V_\pi(B) = \frac{1}{2}$

3rd: $V_\pi(A) = 0$, $V_\pi(B) = \frac{2}{3}$

4th: $V_\pi(A) = 0$, $V_\pi(B) = \frac{3}{4}$

5th: $V_\pi(A) = 0$, $V_\pi(B) = \frac{4}{5}$.

Now update $V_\pi(A)$ using

bootstrapping: $V_\pi(A) = 0 + \frac{4}{5} = \frac{4}{5}$.

Model from data?

$$E_n = (V_B - 0)^2 + (V_B - 1)^2 + (V_A - 0)^2$$

$$2V_B + 8(V_B - 1) = 0$$

$$2V_A = 0$$

$$V_B = \frac{4}{5}$$

$$V_A = 0$$

TD Control

- SARSA
- Q-Learning

$\downarrow \downarrow \downarrow \downarrow \downarrow$ $\rightarrow$ On-policy
 $A_{t+1} \sim \epsilon$ -greedy

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \left(R_{t+1} + \underbrace{V \frac{\epsilon}{|A(S_t)|}}_{\downarrow} Q(S_t, A_{t+1}) - Q(S_t, A_t) \right)$$

$1 - \epsilon + \frac{\epsilon}{|A(S_t)|}$ argmax $Q(S_t, a)$
 $\frac{\epsilon}{|A(S_t)|}$ as $a \in A(S_t)$
 any other action.

SARSA: On-Policy TD Control

- State, Action, Reward, State, Action

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$$

- On-policy
 - Exploration using ϵ -greedy
- As always, we have the two aspects of GPI

SARSA

- One-step bootstrapping for updating action-value function

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

- Bootstrap using an improved ϵ -greedy policy

$$A_{t+1} \sim \pi(a|S_{t+1}) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(S_{t+1})|} & a \in \arg \max_j Q(S_{t+1}, j) \\ \epsilon/|A(S_{t+1})| & a \in A(S_{t+1}) \end{cases}$$

SARSA

- 1: Initialize $Q(s, a), \forall (s, a)$
- 2: **while** true **do**
- 3: Initialize step $t = 0$ at beginning of a new episode
- 4: Initial state S_0 , action A_0 , reward R_1
- 5: **for** step t in an episode **do**
- 6:

Policy improvement that results from previously updated Q

Policy Eval

$$A_{t+1} \sim \pi(a|S_{t+1}) = \begin{cases} 1 - \epsilon & a \in \arg \max_j Q(S_{t+1}, j) \\ \epsilon / |A(S_{t+1})| & a \in A(S_{t+1}) \end{cases}$$

- 7: $Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$
- 8: Agent executes action A_{t+1} , gets R_{t+2} , and the new state is S_{t+2}
- 9: $t = t + 1$

Q-Learning: Off-Policy TD Control

- State, Action, Reward, State

$$S_t, A_t, R_{t+1}, S_{t+1}$$

- Learn the optimal policy
- Off-policy
 - Behavior policy uses ϵ -greedy to generate the episode
 - Bootstrapping assuming the current optimal (greedy action selection) policy

Q-Learning

- Since we are interested in the optimal policy
 - One-step bootstrapping chooses greedy action

$$a_{t+1}^* = \arg \max_{a \in A(S_{t+1})} Q(S_{t+1}, a)$$

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, a_{t+1}^*) - Q(S_t, A_t))$$

- Encourage exploration

$$A_{t+1} \sim \pi(a|S_{t+1}) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(S_{t+1})|} & a \in \arg \max_j Q(S_{t+1}, j) \\ \epsilon / |A(S_{t+1})| & a \in A(S_{t+1}) \end{cases}$$

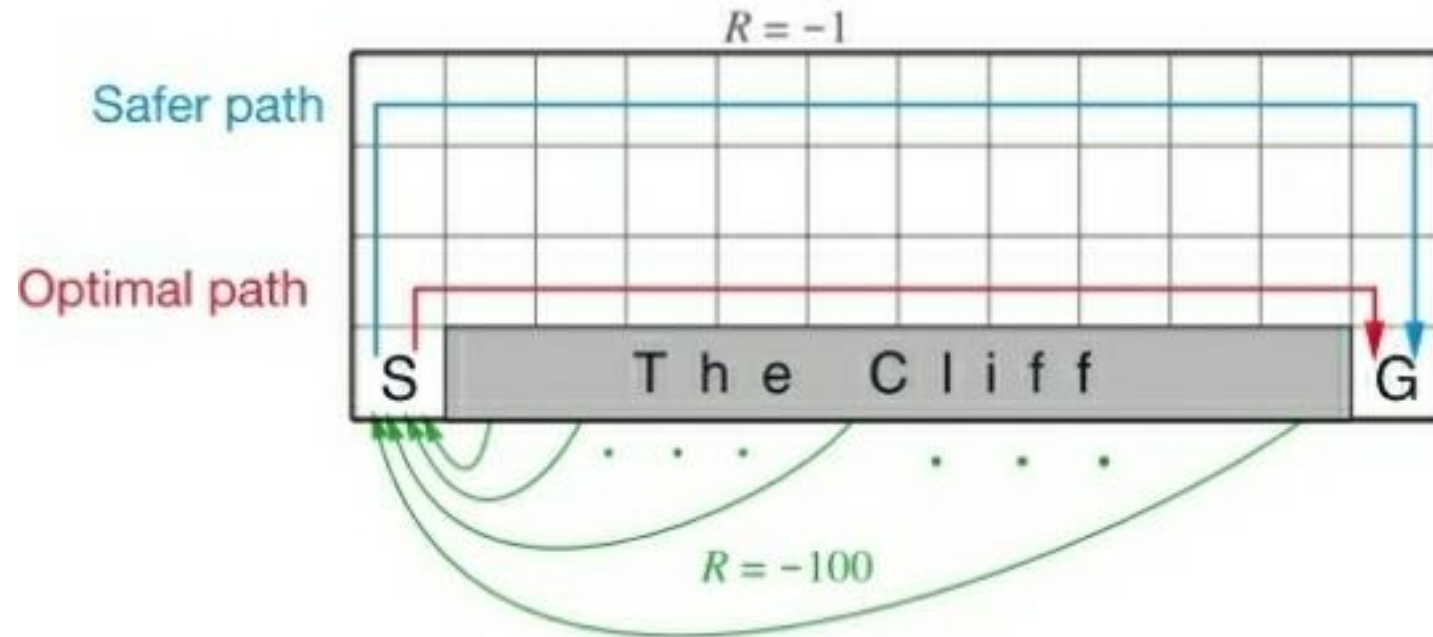
Q-Learning

```

1: Initialize  $Q(s, a), \forall (s, a)$ 
2: while true do
3: Initialize step  $t = 0$  at beginning of a new episode
4: Initial state  $S_0$ , action  $A_0$  using  $\epsilon$ -greedy, reward  $R_1$ 
5:   for step  $t$  in an episode do
6:      $Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a \in A(S_{t+1})} Q(S_{t+1}, a) - Q(S_t, A_t))$ 
7:     Agent chooses  $A_{t+1}$  using  $\epsilon$ -greedy policy
8:      $t = t + 1$ 

```

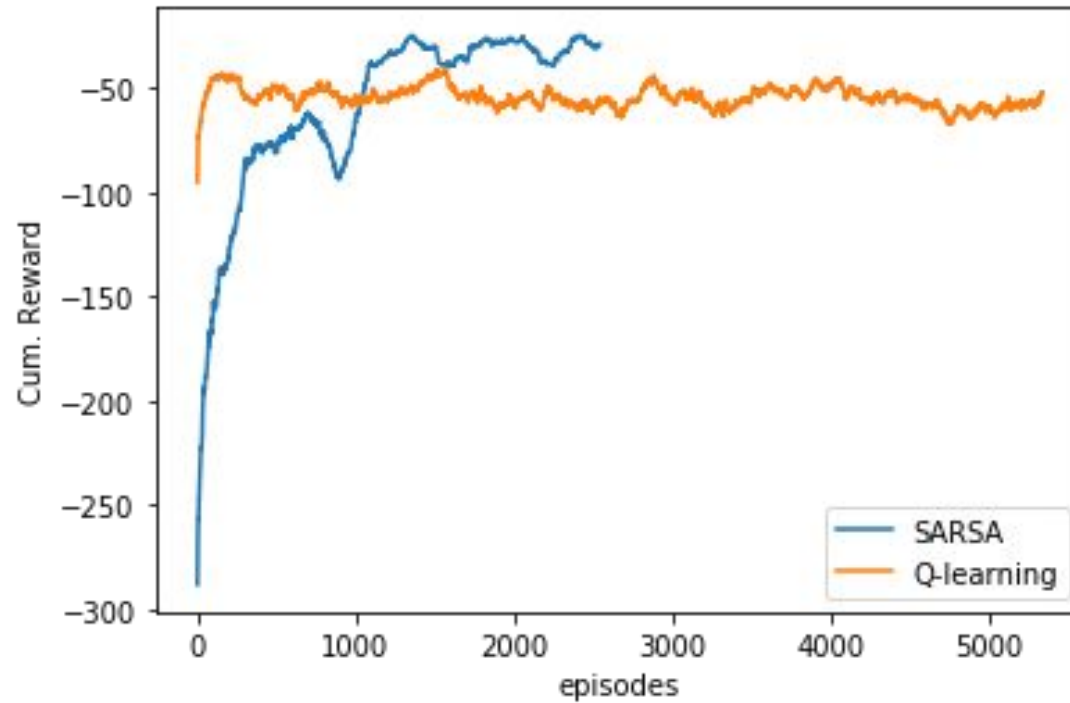
Cliff Walking: SARSA vs. Q-Learning



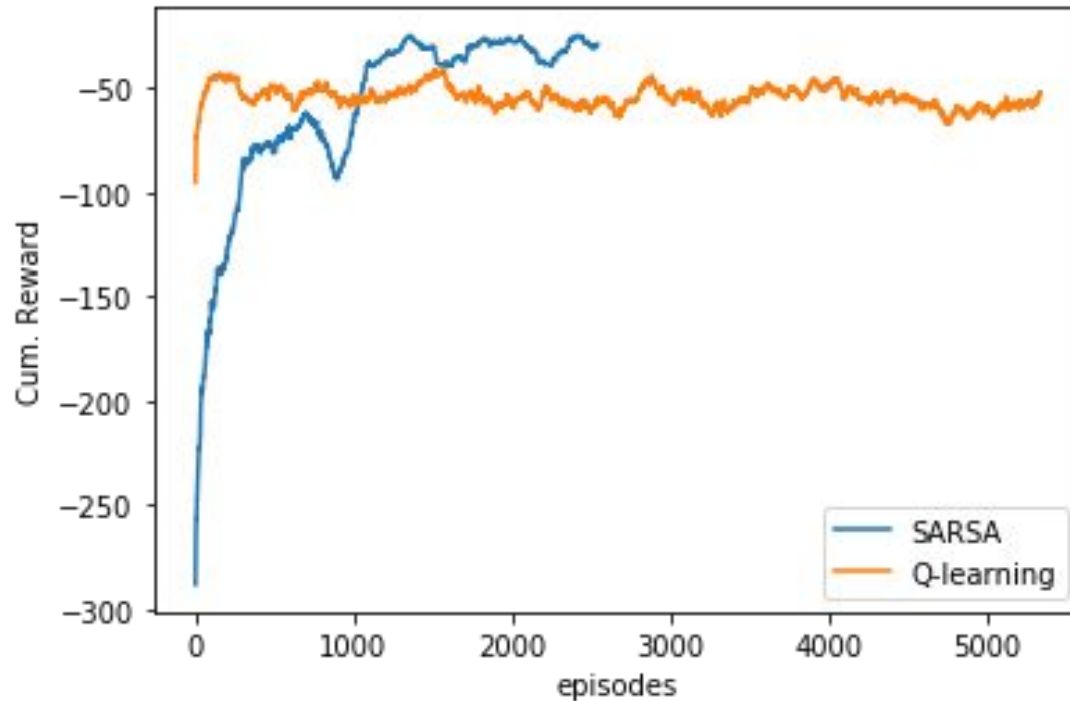
<https://towardsdatascience.com/reinforcement-learning-cliff-walking-implementation-e40ce98418d4>

Example 6.6 in <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>

Cliff Walking: SARSA vs Q-Learning



Cliff Walking: SARSA vs Q-Learning



- SARSA learns the best ϵ -greedy policy
- Such a policy will learn to stay away from the cliff
- Q-learning will learn the optimal policy
- Suffers because behavior policy is ϵ -greedy

Expected SARSA

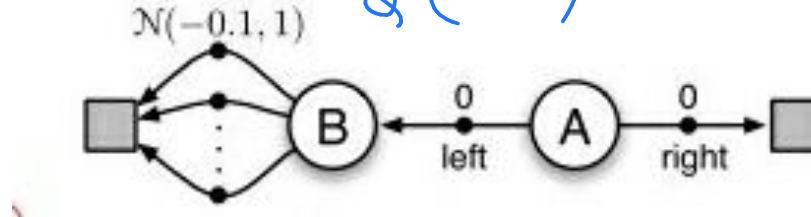
- Updating Q-values

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha (R_{t+1} + \underset{A_{t+1}}{E}[Q(S_{t+1}, A_{t+1})] - Q(S_t, A_t))$$

Expected SARSA

- Move Q-values deterministically in the direction as SARSA does in expectation
- Can be Off-Policy
- Both SARSA and Q-learning are instances of Expected SARSA

Maximization Bias



- Suppose during an episode taking action left in State B gets a large positive reward
- Impacts action-value of (State A, action left)
- Together with greedy (maximizing) action selection may mean left will be chosen in state A for a long time
 - Till many steps later $Q(B, \text{left})$ gets updated to a value closer to average -0.1

Image: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>

Maximization Bias

- Agents actions are biased by our currently maximum action-value estimates
- May slow down reaching the true optimal action-value estimates
- Alleviate bias by maintaining more than one independent valuations of (s,a)
 - Choose your greedy action but when bootstrapping value it as per the other valuation

Double Q-learning

- Maintain independent Q -value tables $Q(s,a)$ and $Q'(s,a)$ for all (s,a)
- The updates for the independent tables are:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q'(S_{t+1}, \arg \max_{a \in A(S_{t+1})} Q(S_{t+1}, a)) - Q(S_t, A_t))$$

$$Q'(S_t, A_t) = Q'(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, \arg \max_{a \in A(S_{t+1})} Q'(S_{t+1}, a)) - Q'(S_t, A_t))$$

n-Step Bootstrapping

- Frequent actions and infrequent bootstrapping
- A middle ground between MC and 1-step TD

n-Step TD Prediction

n-Step SARSA

n-Step Expected SARSA

Approximate Solution Methods

Approximate Solution Methods

- Computationally infeasible to treat all states/ state-action pairs separately
- Generalize
 - Use experience over a limited subset of state space to approximate well a much larger set of states
- We want to approximate the state value and action value functions
 - No training set, however
 - The true values are not known
 - Must use noisy values obtained via experience

Generalize

Instead of a tabular representation:

$$x = 1, 2, 3, \dots, 10^{12}$$

$$y = 1, 4, 9, \dots, 10^{24}$$

Use a concise functional approximation:

$$y = x^2$$

Function Approximation

- Curve fitting
- Machine learning
- Artificial Neural Nets
- Training data set is available

Function Approximation

- We want to approximate a state-value function or an action-value function
- However, the true state-value and action-value functions are NOT known for any set of values
 - No training set available
- We must work with noisy estimates of these functions while we learn the approximation

Function Approximation

- We must work with noisy estimates of these functions while we learn the approximation
 - Use MC, TD to come up with noisy estimates in an online manner!

Value Function Approximation

- We want to find the parameter vector $\mathbf{w}$ such that

$$v_{\pi}(s) \approx \hat{v}(s, \mathbf{w})$$

- The function approximation is

$$\hat{v}(s, \mathbf{w})$$

- The parameter vector $\mathbf{w}$ is typically such that

$$\mathbf{w} \in R^d, d \ll |S|$$

Value Function Approximation

- The value function must give values for $|S|$ states

$$v_{\pi} \in \mathbb{R}^{|S|}$$

- If functional approximation is a linear function of $\mathbf{w}$
- We are essentially projecting a vector in $|S|$ dimensional space onto a d -dimensional space

Feature matrix.

$$v_{\pi} \approx \Phi_{|S| \times d} \mathbf{w}_{d \times 1}$$

Value Function Approximation

- More generally, we could have a non-linear approximation architecture
 - For example, a neural net with weights $\mathbf{w}$

Value Function Approximation

- Prediction Objective

- Find the vector $\mathbf{w}$ that minimizes the value error

$$VE = E_{\pi}[(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2]$$

$$= \sum_s \mu(s) (v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2$$

- On $\mu(s)$

Write $\mu(s)$ in terms of $\mu(s')$, $s' \in S$.

Write $P[S_t = s]$ in terms of $P[S_{t+1} = s']$, $s' \in S$.

$P(s', a | s, a)$
 $\pi(a|s)$

$\lim_{t \rightarrow \infty} P[S_t = s]$

Value Function Approximation

- Prediction Objective
 - Find the vector $\mathbf{w}$ that minimizes the value error

$$\begin{aligned} \text{VE} &= E_{\pi}[(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2] \\ &= \sum_s \mu(s)(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2 \end{aligned}$$

- Set the gradient wrt $\mathbf{w}$ to _?

Value Function Approximation

- Prediction Objective
 - Find the vector $\mathbf{w}$ that minimizes the value error

$$\begin{aligned}\text{VE} &= E_{\pi}[(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2] \\ &= \sum_s \mu(s)(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2\end{aligned}$$

- Set the gradient wrt $\mathbf{w}$ to 0?
- But we don't know $\mu(s)$ and $v_{\pi}(s)$

Value Function Approximation

$$\begin{aligned} \text{VE} &= E_{\pi}[(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2] \\ &= \sum_s \mu(s)(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2 \end{aligned}$$



- Exact Expected Squared Error is Unavailable
- Use a (noisy) sample estimate

What would you propose?

$$(v_{\pi}(s_t) - \hat{v}(s_t, \mathbf{w}))^2$$

Replace by a
sample target.

Value Function Approximation

$$\begin{aligned} \text{VE} &= E_{\pi}[(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2] \\ &= \sum_s \mu(s)(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2 \end{aligned}$$

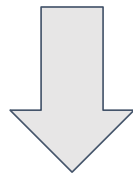


- Exact Expected Squared Error is Unavailable
- Use a (noisy) sample estimate

$$(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2$$

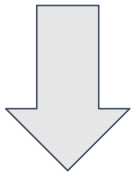
Value Function Approximation

$$\begin{aligned} \text{VE} &= E_{\pi}[(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2] \\ &= \sum_s \mu(s)(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2 \end{aligned}$$



- Exact Expected Squared Error is Unavailable
- Use a (noisy) sample estimate

$$(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2$$



- But we don't know $v_{\pi}(s)$
- Replace it by a target we can calculate online!

$$(U(s) - \hat{v}(s, \mathbf{w}))^2$$

Value Function Approximation

- Find the vector $\mathbf{w}$ that minimizes a noisy estimate of

$$\begin{aligned}\text{VE} &= E_{\pi}[(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2] \\ &= \sum_s \mu(s)(v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2\end{aligned}$$

- Use stochastic gradient descent

Value Function Approximation Using MC

- Just as we updated $Q(s,a)$ and $V(s)$ using episodes, we want to update the vector $\mathbf{w}$
- Step updates of $\mathbf{w}$ in the direction of -ve gradient
- At time step t the gradient is

$$\nabla_{\mathbf{w}_t} (G_t - \hat{v}(S_t, \mathbf{w}_t))^2$$

(Handwritten underline and dimension $d \times 1$)

Value Function Approximation Using MC

- Step updates of $\mathbf{w}$ in the direction of -ve gradient
- At time step t the gradient is

$$-2(G_t - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}_t} v(S_t, \mathbf{w}_t)$$

- Stochastic gradient step update at time t

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (G_t - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}_t} v(S_t, \mathbf{w}_t)$$

Gradient MC Prediction

```

1: Initialize  $\mathbf{w}$ 
2: while true do
3: Randomly pick an initial state  $S_0$ 
4: Generate an episode using  $\pi$ . We get
5:    $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T, R_T$ 
6:    $G = 0$ 
7:   for  $t = T - 1, T - 2, \dots, 0$  do
8:      $G = G + \gamma R_{t+1}$ 
9:      $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (G_t - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}_t} v(S_t, \mathbf{w}_t)$ 

```
